Ayudantía 2

SISTEMAS DISTRIBUIDOS

PROFESOR: NICOLAS HIDALGO

AYUDANTE CÁTEDRA: VICENTE DÍAZ, JOAQUÍN VILLEGAS

AYUDANTE CORRECTOR: BENJAMÍN ACEITUNO, ISIDORA GONZALEZ, NATALIA ORTEGA

Contacto

Benjamín Aceituno
benjamin.aceituno@mail.udp.cl
Vicente Díaz
Vicente.diaz5@mail.udp.cl
Isidora González
isidora.gonzalez4@mail.udp.cl
Natalia Ortega
natalia.ortega1@mail.udp.cl
Joaquín Villegas
joaquin.villegas@mail.udp.cl

Si estas en windows necesario instalar WSL(compilador de linux):

1) Buscar Windows PowerShell y ejecutar como administrador

2) Ejecutar el comando: wsl --install
(si hay un error se debe activar la vistualizacion desde la BIOS)
https://support.microsoft.com/es-es/windows/habilitar-la-virtualización-en-windows-c5578302-6e43-4b4b-a449-8ced115f58e1

3) Reiniciar PC.

4) Una vez reiniciado se abrirá una consola de ubuntu donde tendran que crear un usuario y contraseña.

5) Entrar a Panel de control < Desinstalar un programa < Activar o desactivar caracteristicas de Windows
Una vez en este lugar buscar "Subsistema de Linux para Windows" y activarlo.

6) Descargar Docker (Si es windows, opcion AMD64)(Si es linux empezar tutorial desde aquí)
https://www.docker.com

7) Reiniciar.

8) Una vez instalado todo comprobar que todo este instalado correctamente.
Abrir WindowsPowerShell y ejecutar: wsl --status

Si les aparece un mensaje asi es correcto ya que docker ocupa la version 2:
Default Distribution: Ubuntu
Default Version: 2

9) Ahora para comprobar el funcionamiento de Docker deben entrar a WSL:
docker --version
Si les aparece un mensaje asi es correcto:
Docker version 25.0.2, build ...

Si les aparece un mensaje asi:
The command 'docker' could not be found in this WSL 2 distro.
Deben ir a Docker Desktop < Settings < Resources < WSL integration < Enable integration with additional distros < Ubuntu: ON

Por último ejecutar: docker run hello-world
Si ven el mensaje: Hello from Docker!
Está instalado correctamente.

Tutorial instalación Docker (Windows)

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15
    restart: always
    ports:
      - "5433:5432"
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: yahoo_dataset
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init_db.sql:/docker-entrypoint-initdb.d/init_db.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d yahoo_datase
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - yahoo_network

  pgadmin:
    image: dpage/pgadmin4
    restart: always
    ports:
      - "5050:80"
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@admin.com
      PGADMIN_DEFAULT_PASSWORD: admin
    volumes:
      - pgadmin_data:/var/lib/pgadmin
    depends_on:
      - postgres
    networks:
      - yahoo_network
```

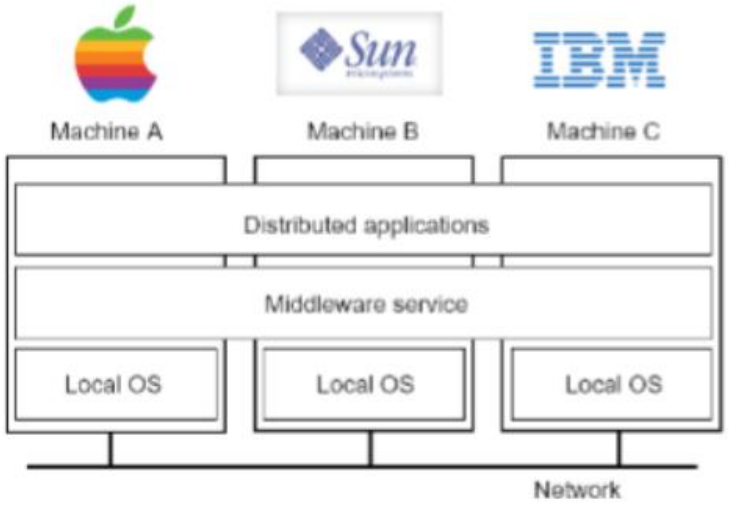
```
Para levantar todos los dockers en la carpeta raiz se debe ejecutar:

(Levantamiento lento)
En caso de que sea necesario reconstruir las imagenes de cada servicio
ya que hubo un cambio en el Dockerfile o cambiaste las dependencias o
quieres asegurarte de que la imagen este actualizada, utilizar:

docker-compose up --build

(Levantamiento rápido)
En caso de que modifiques el código fuente y estas desarrollando, lo
mejor es levantar contenedores ya creadas en tu máquina, no recompila
gracias a los volumenes, utilizar:

docker-compose up
=====================================================================
Para visualizar si los dockers están funcionando comprobarlo con:
docker ps
o
docker ps -a
=====================================================================
```

Utilización de Docker

←docker-compose.yml

¿Qué son los sistemas distribuidos?

- Conjunto de máquinas que coordinan sus acciones y que aparecen al usuario como sistema único y coherente.
- Middleware: SD emplea software sobre la red para lograr cometido.
- Red de computadores: usuarios interactúan directamente con las máquinas.



¿Porqué y cuándo distribuir?

- Lograr escalabilidad, disponibilidad, eficiencia y cercanía al usuario.
- Supera limitaciones de un sistema centralizado.
- Dependencia al tamaño e infraestructura.

Ventajas:

- Compartir recursos.
- Mejora de desempeño.
- Mayor adaptabilidad.
- Mayor disponibilidad.
- Escalabilidad.

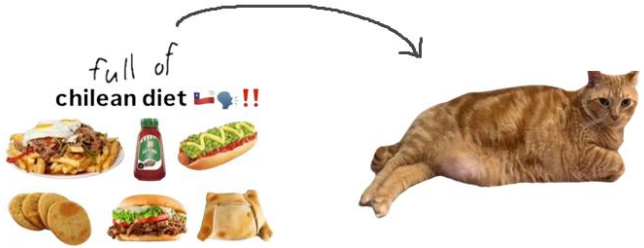
Desafíos:

- Heterogeneidad del hardware.
- Consistencia de la interacción.
- Siempre disponible.
- Usuarios y apps no deben notar caídas.
- Seguridad

Objetivos de diseño

Existen 4 objetivos al diseñar un sistema distribuido:

1) Poner recursos a disposición.
2) Hacer transparente la distribución.
3) Ser abierto.
4) Escalabilidad.



1) Poner recursos a disposición.

- Facilitar a los usuarios y aplicaciones acceder recursos remotos de manera eficiente.
- Razones para compartir:
  - Económicas.
  - Colaboración e intercambio de información.
- Si se incrementar conectividad y compartición de recursos la seguridad se vuelve relevante.
  - Seguridad de la comunicación.
  - Seguridad de acceso a los datos.

2) Hacer transparente la distribución.

- Esconder el hecho que procesos y recursos están físicamente distribuidos en múltiples computadores.
- Usuarios y aplicaciones ven un sistema único.
- Sistema coherente.

- Transparencia respecto a diferentes ámbitos:
  - Acceso.
  - Ubicación.
  - Migración.
  - Re-localización.
  - Replicación.
  - Concurrencia.
  - Falla.

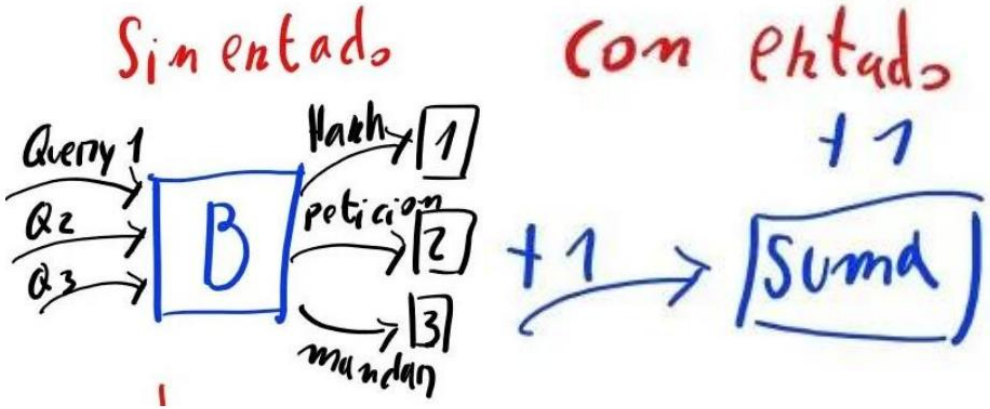
3) Ser abierto

- Sistema distribuido abierto es aquel que ofrece servicios bajo reglas estándar (sintaxis, semántica).
- Interface Definition Language (IDL), lenguaje de especificación de componentes del sistema, como, por ejemplo:
  - Advanced Programming Interface (API).
  - Llamadas a procedimientos remotos (RPC).
- Interoperabilidad: permite que procesos se comuniquen y generen implementaciones diferentes.
- Portabilidad: extensiones pueden ser usadas en otros sistemas
- Fácil extensión: agregar más componentes
  - En la práctica, muchos sistemas son monolíticos (cerrados).

4) Escalabilidad

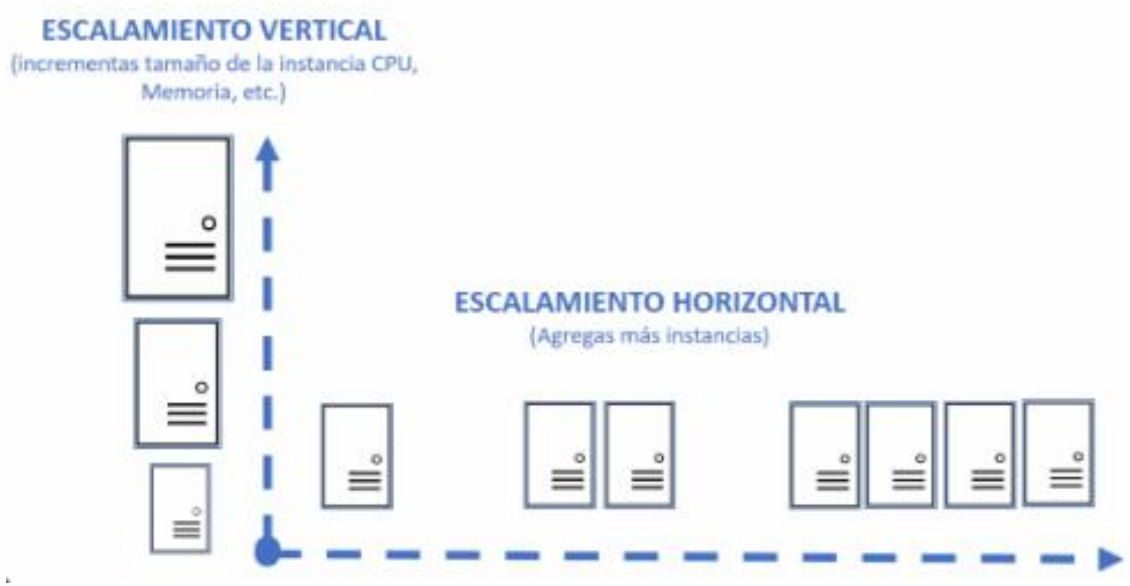
Existen 2 tipos:

- Escalabilidad Horizontal (scaling out): Aumentar la cantidad de los recursos.
  - Agregar más nodos de trabajo.
  - Front-End sin estado -> lo que haya pasado antes no influye en las próximas decisiones.
  - Backend con particionamiento, modelo master/slave.



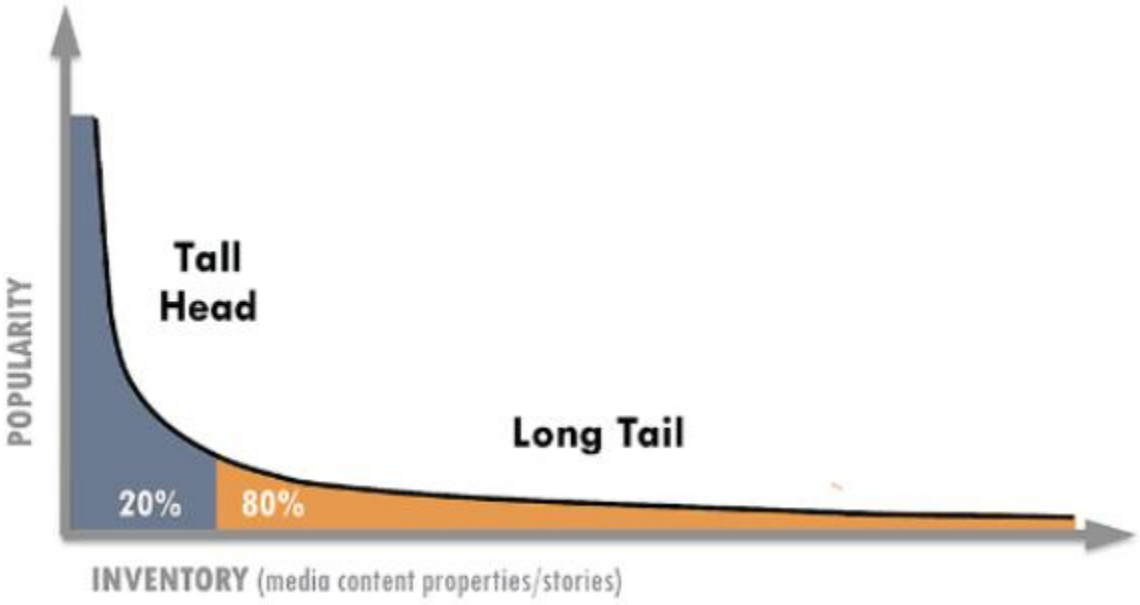
4) Escalabilidad

- Escalabilidad Vertical (scaling up): Aumentar las capacidades de mi recurso
  - Agregar CPU, Memoria, disco.
  - Manejar simultáneo de más conexiones, operaciones, usuarios.
  - Buen I/O y modelos de concurrencia (hebras, asincronismo, etc...).



4) Escalabilidad

- Existen 3 técnicas básicas para escalar:
  I. **Particionar y distribuir:** dividir un componente y distribuirlo, el cuál puede tener 2 variaciones:
     a. **Particionamiento Vertical**
     b. **Particionamiento Horizontal (Sharding)**
  II. **Replicar:** hacer copiar de los recursos en diferentes lugares.
  III. **Cache:** subconjunto de recursos de acceso fácil y rápido.



I) Particionar y distribuir

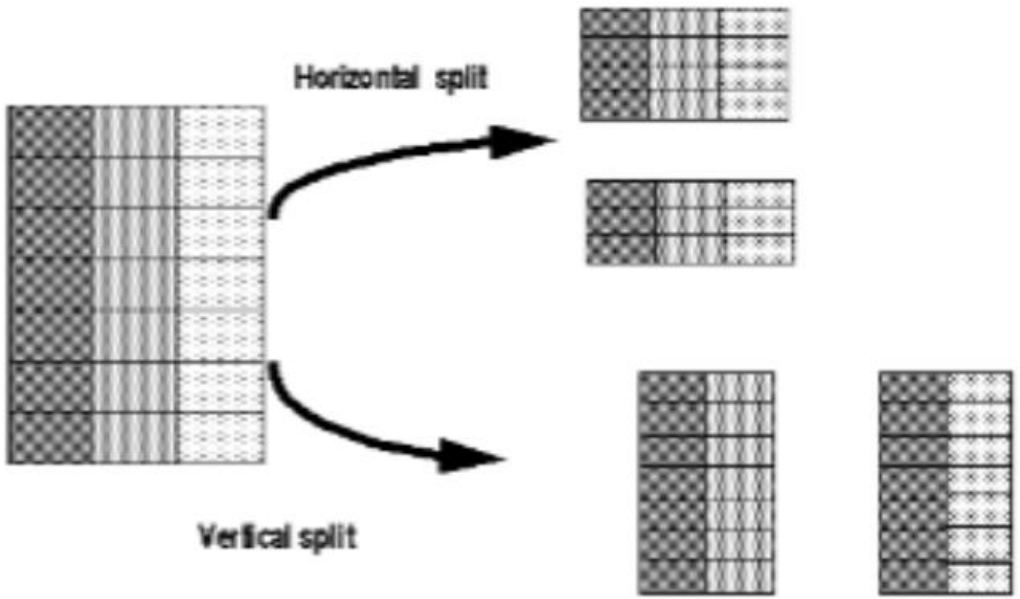
Particionamiento Horizontal (Sharding):

i. **Hash-Based:** función de hash se aplica a atributos de datos para determinar a qué shard pertenece.

ii. **Range:** datos particionados (ordenados) por un número o un rango determinado de datos.

iii. **Geographical:** datos se ubican a través de diferentes ubicaciones geográficas o centros de datos.

iv. **Directory o Virtual Bucket:** utilización de un servicio de lookup centralizado o directorio.



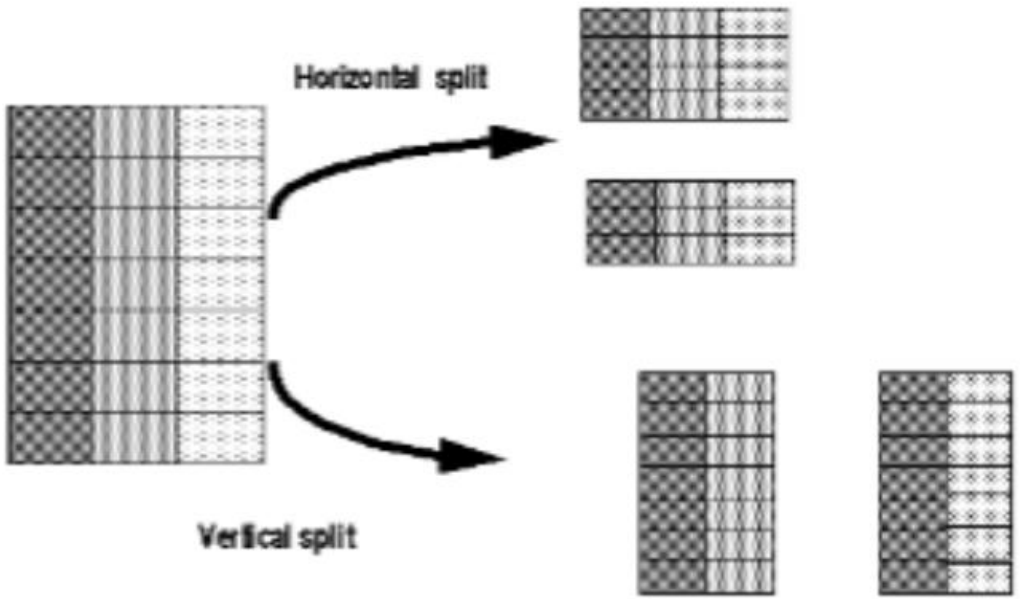
II) Replicar

Copia de un recurso que reside en un lugar diferente.

- **Permite tolerar fallos:** evitar puntos únicos de fallos.
- **Permite accesos paralelos:** evita cuellos de botella.
- **Balance de carga:** distribuye carga entre las réplicas.

Desventaja:

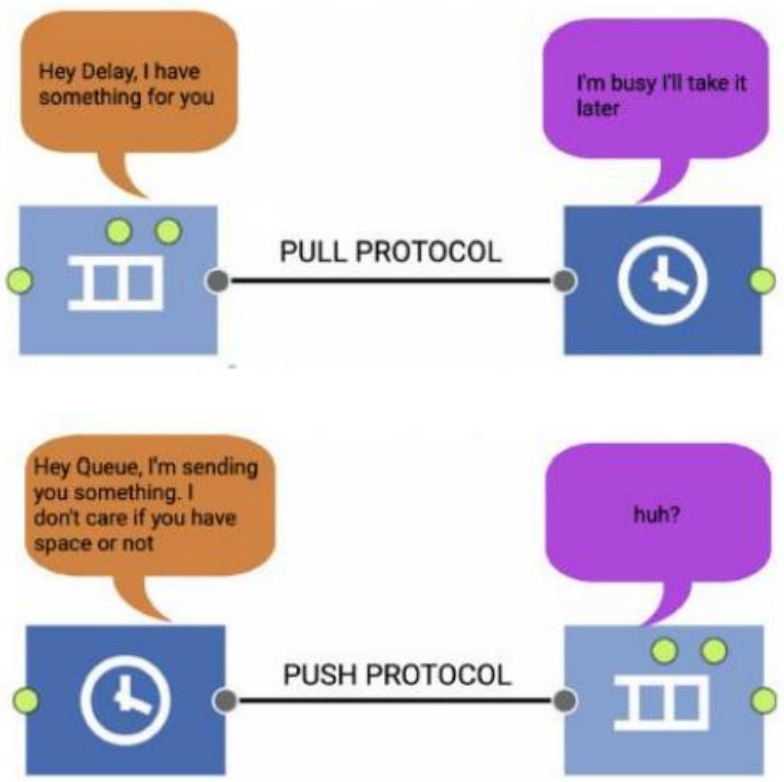
- Costo de la réplica.
- Mantenimiento estado: costo depende de número de réplicas y relación read/write del sistema.



II) Replicar

Problemas:

- Ubicación del servidor de la réplica:
  - Trade-off Performance
  - Elasticidad: agregar/reducir réplicas
- Dónde se pone la réplica
- Actualización de la réplica:
  - Estado
  - Updates síncronos /asíncronos
- ¿Quién inicia la actualización?
  - **Push-based protocols:** replica propaga updates.
  - **Pull-based protocols:** replica pregunta si hay updates, polling.



PUSH

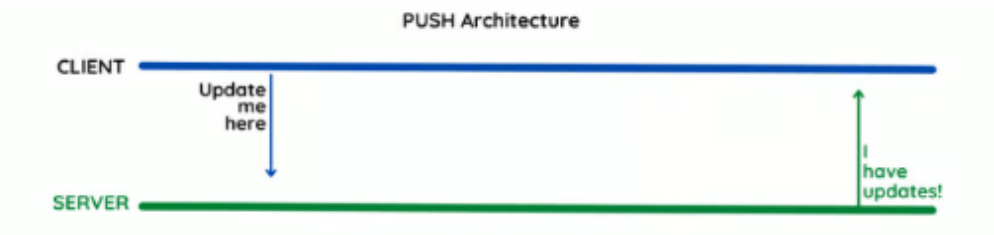
Cuando un estado cambia, el sistema realiza un push y notifica a las réplicas de la actualización

Resulta mas rápido que las réplicas se actualicen

Costo innecesario sin demanda real

Saturación de la red ante la presencia de demasiadas actualizaciones

Ejemplos: mail, bases de datos relacionales

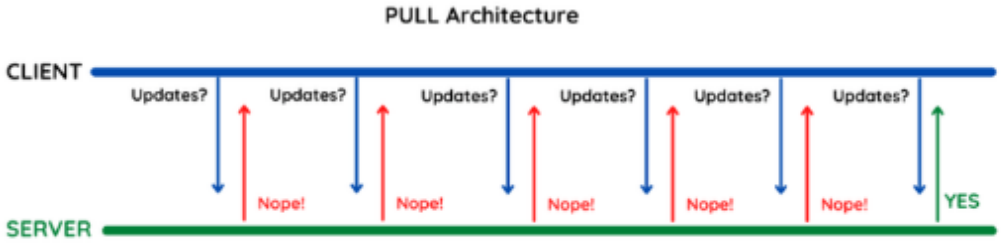


PULL

Las réplicas conslultan si es que hay actualizaciones en los estados

Si las réplicas están muy ocupadas, pueden perder updates, generando inconsistencia

Ejemplos: Motores de búsqueda (google, yahoo), sistemas de archivos como HDFS



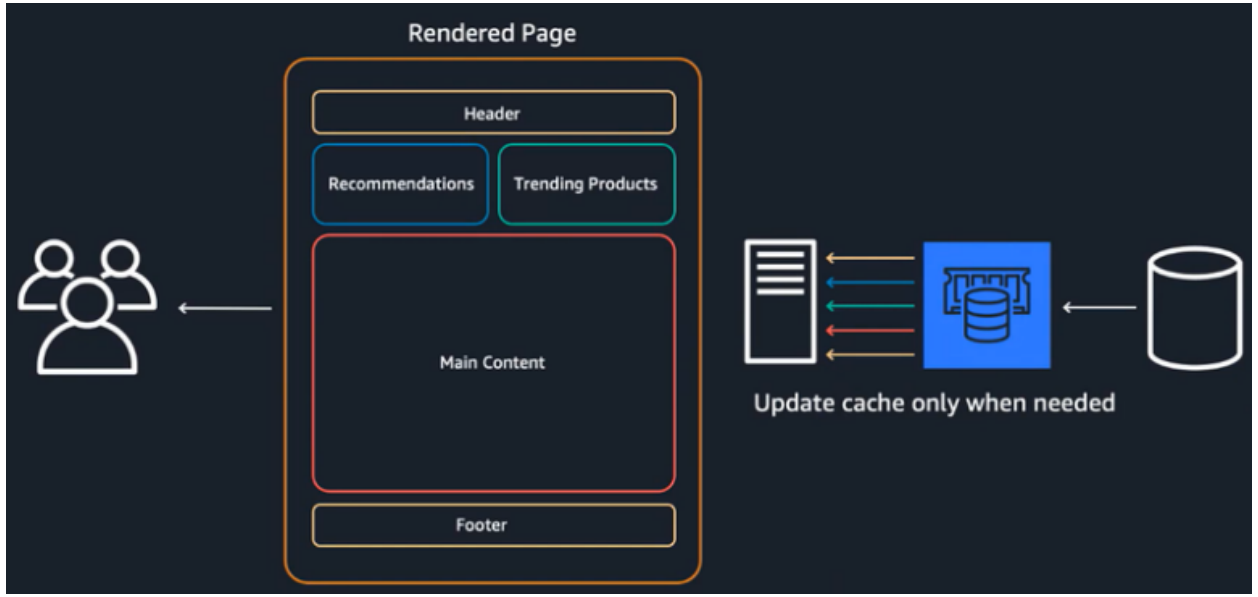
II) Replicar

III) Cache

Busca reducir latencia y carga del backend, optimiza uso de recursos. Se basa en:

a) **Localidad temporal:** Los datos usados recientemente tienen alta probabilidad de volver a ser utilizados pronto.

b) **Localidad espacial:** Si se accede a un dato, es probable que se acceda también a los datos cercanos a él.



III) Cache

Se tratra de un duelo entre Hit vs Miss. Necesitamos mecanismos rápidos y eficientes en la selección de lo que mantendremos en cache:

Políticas de remoción

Apuntan a captar el patrón de repetición de las solicitudes. Debe ser simple y rápido.

1. LRU
2. FIFO
3. RANDOM
4. MRU
5. SIZE

No se utiliza con frecuencia LFU ya que es muy costoso

Políticas de ingreso

- Mecanismos de control de ingreso ¡No todo sirve! Ley de potencias.
- Para determinar que ingresa y que no al cache, se hace un análisis histórico.
- El histórico solo sirve para sistemas con comportamientos más estacionarios, de lo contrario no resulta útil.

TTL

- Es el tiempo que determina cuanto se mantiene una entrada en cache.
- Una vez que expira el tiempo, se elimina del cache automáticamente.
- Ayuda a controlar la obsolescencia.

Sistema cache de dos niveles

Estático

- Contiene los resultados de las consultas más frecuentes del pasado.
- Estas entradas están bloqueadas (no se reemplazan dinámicamente) y se actualizan periódicamente según los datos de uso.

Ventajas:

- Consultas muy populares se pueden resolver rápido.
- Se accede sin sincronización, lo que mejora el rendimiento con múltiples hilos.
- Ejemplo: consultas frecuentes a un buscador web que no cambian mucho con el tiempo.

Dinámico

- Contiene entradas que se gestionan con una política de reemplazo.
- Atiende consultas populares solo temporalmente o que surgen por eventos inesperados (ej: noticias de último minuto).
- Se elige qué borrar cuando hay un cache miss y el conjunto dinámico está lleno.
- Políticas de reemplazo soportadas: LRU, LRU/2, FBR, SLRU, 2Q, PDC.
- La elección de la política se hace al iniciar el sistema y afecta solo al conjunto dinámico.

Dinámica con una consulta

1. Se busca q en el conjunto estático.
   - Si está → cache hit, se devuelve inmediatamente.
2. Si no está, se busca en el conjunto dinámico.
   - Si está → cache hit.
   - Si no está → cache miss → se consulta al backend (WSE) y se puede reemplazar una entrada del conjunto dinámico según la política elegida.

Ventajas

1. El conjunto estático asegura acceso rápido a las consultas más frecuentes.
2. El conjunto dinámico permite cubrir picos de popularidad o consultas inesperadas.
3. Mejora throughput y eficiencia, especialmente en sistemas con muchos hilos concurrentes.

III) Cache

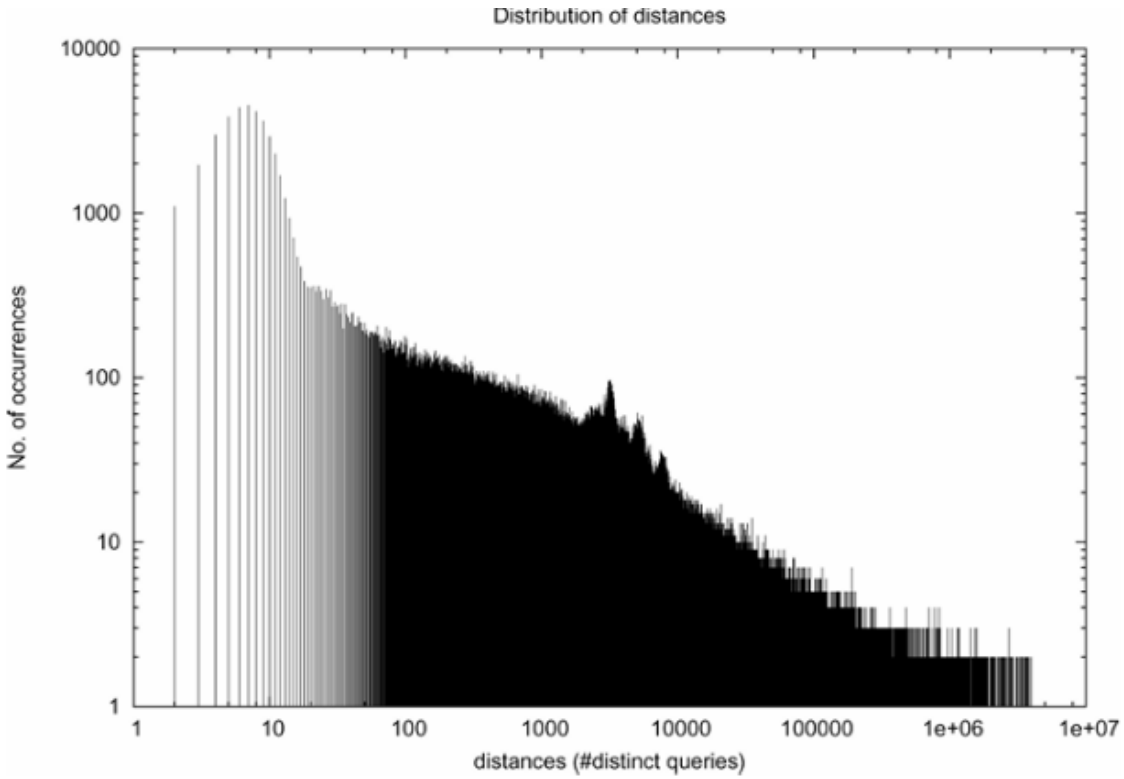
Problema

Suponga que trabaja con un motor de búsqueda como Alta Vista, que recibe millones de consultas de usuarios. Analizando los logs de consultas observa que existen 128000 consultas populares, entre estas hay consultas populares durante largos periodos de tiempo y otras solo en cortos picos de tiempo. El cache del servidor tiene espacio limitado y no puede almacenar todas las consultas, por lo que es su trabajo encontrar el mejor sistema de cache para Alta Vista.

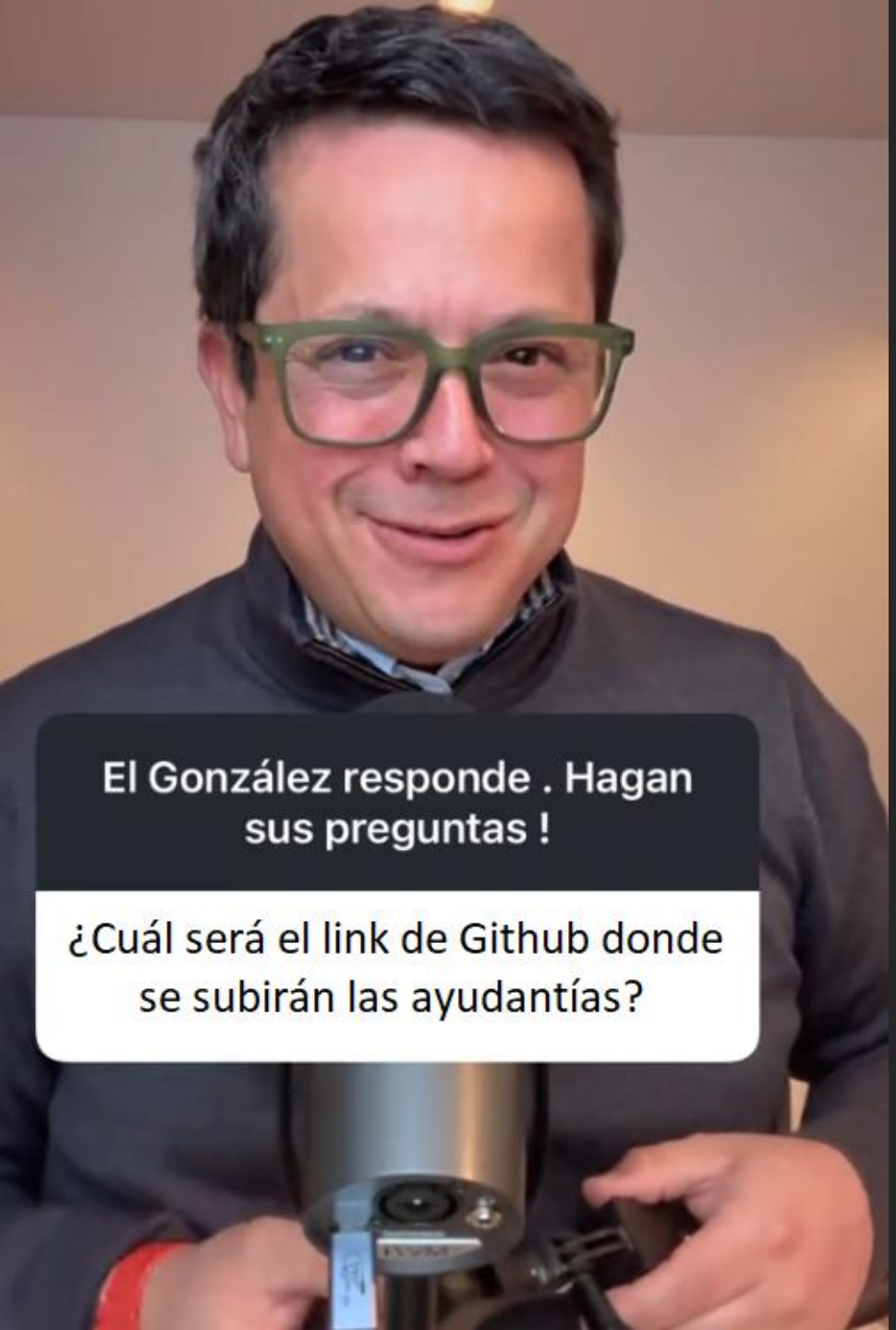
Como buen ingeniero, antes de tomar cualquier decisión usted decide hacer un estudio de la cantidad de consultas no populares (distancia) que se hacen antes de que una query popular vuelva a repetirse, por ejemplo, si A es una consulta popular y se tiene A, B, C, A. Entonces la distancia es de 2. Los hallazgos los presentó en el gráfico de la derecha.

Con el contexto, el análisis del gráfico y lo que ha aprendido del curso, determine:

1) Dado que el caché es limitado, ¿qué política de reemplazo sería más adecuada: LRU, LFU o una híbrida? ¿Por qué?
2) Si se aumenta el tamaño del caché, ¿hasta qué punto se obtienen mejoras antes de que la ganancia sea marginal?



Distancia = numero de consultas distintas que llegaron entre dos envíos de la misma consulta frecuente



Link Github

Repositorio:

https://github.com/VicenteDiazH/AY-SD2026-1-S3